

DBMS -- JOINS & ER Model

Target: Name all JOIN types with one-line difference. Draw a basic ER diagram verbally.

JOIN Types -- All Six

Reference tables used throughout:

Employees: E1-Alice(D1), E2-Bob(D2), E3-Carol(no dept)

Departments: D1-HR, D2-Finance, D3-IT(no employees)

JOIN Type

Returns

Result from example

INNER JOIN

Only rows that match in both tables

Alice-HR, Bob-Finance

LEFT JOIN

All rows from left + matched from right (NULL if no match)

Alice-HR, Bob-Finance, Carol-NULL

RIGHT JOIN

All rows from right + matched from left (NULL if no match)

Alice-HR, Bob-Finance, NULL-IT

FULL OUTER JOIN

All rows from both, NULLs where no match

Alice-HR, Bob-Finance, Carol-NULL, NULL-IT

CROSS JOIN

Every row every row (cartesian product)

3 employees 3 depts = 9 rows

SELF JOIN

Table joined with itself

Employee-Manager in the same table

JOIN Queries -- Side by Side

```
```sql
```

```
-- Sample tables
```

```
-- employees: emp_id, name, dept_id
```

```
-- departments: dept_id, dept_name
```

```
-- INNER JOIN -- only matched rows
```

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

```
-- LEFT JOIN -- all employees, even without a department
```

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

```
-- RIGHT JOIN -- all departments, even empty ones
```

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

```
-- FULL OUTER JOIN -- everything from both sides
```

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

```
-- CROSS JOIN -- every combination
```

```
SELECT e.name, d.dept_name
```

```
FROM employees e
```

```
CROSS JOIN departments d;
-- SELF JOIN -- find each employee's manager (manager_id is in same table)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
````
```

FULL OUTER JOIN vs CROSS JOIN -- The Most Confused Pair

They look similar because both "include everything" -- but the mechanism is completely different.

Same Tables Used for Both
employees (3 rows):

| emp_id | name | dept_id |
|--------|------|---------|
|--------|------|---------|

| | | |
|---|-------|----|
| 1 | Alice | 10 |
|---|-------|----|

| | | |
|---|-----|----|
| 2 | Bob | 20 |
|---|-----|----|

| | | |
|---|-------|------|
| 3 | Carol | NULL |
|---|-------|------|

departments (3 rows):

| dept_id | dept_name |
|---------|-----------|
|---------|-----------|

| | |
|----|----|
| 10 | HR |
|----|----|

| | |
|----|---------|
| 20 | Finance |
|----|---------|

| | |
|----|----|
| 30 | IT |
|----|----|

Alice HR, Bob Finance, Carol has no dept, IT has no employee.

FULL OUTER JOIN Result

```
sql  
SELECT e.name, d.dept_name  
FROM employees e  
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

| name |
|------|
|------|

dept_name

Alice
HR

Bob
Finance

Carol
NULL

NULL
IT

4 rows. Each employee appears once. Each department appears once.
NULL fills in where there is no match on the join condition.

CROSS JOIN Result

```
sql
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

name
dept_name

Alice
HR

Alice
Finance

Alice
IT

Bob
HR

Bob
Finance

Bob
IT

Carol
HR

Carol
Finance

Carol
IT

9 rows (3 employees 3 departments). Every possible pair, no exceptions.
No NULLs from the join. No conditions checked. No matching logic.

Side-by-Side Comparison

Feature

FULL OUTER JOIN
CROSS JOIN

Join condition (ON)

[OK] Required
None

What it does

Match rows; NULL for unmatched
Pair every row with every row

Result rows

$m + n$ (depends on matches)
Always exactly $m \times n$

NULLs in result

[OK] Yes (where no match)
No (no condition to fail)

Result (3 emp, 3 dept)

4 rows
9 rows

Relationship-aware?

[OK] Yes -- uses FK/PK logic
No -- completely blind

Use case

Find unmatched rows on both sides
Generate all combinations (test data, schedules)

The Simplest Mental Model

FULL OUTER JOIN "Smart union" -- match what you can, keep the rest with NULLs

Result = matched pairs + left orphans + right orphans

CROSS JOIN "Brute force" -- no matching, just multiply

Result = EVERY row in A paired with EVERY row in B

Another way to think about it:

- FULL OUTER JOIN is like a sorted guest list with empty seats -- every guest gets a seat, empty seats are marked NULL

- CROSS JOIN is like a round-robin tournament -- every team plays against every other team regardless

Quick Proof They Are Not the Same

Same 3-row tables:

FULL OUTER JOIN 4 rows (3 matched + 1 unmatched dept)

CROSS JOIN 9 rows (3 3 always)

If ALL rows matched perfectly (no NULLs, no orphans):

FULL OUTER JOIN 3 rows (exact 1:1 match)

CROSS JOIN 9 rows (still 3 3)

They only look the same when you talk loosely about "all rows from both tables" --

but FULL OUTER JOIN respects relationships; CROSS JOIN ignores them entirely.

Venn Diagram Mental Model

```
Left table (L)      Right table (R)
  L                  R
only  INNER  only
      (match)
```

LEFT JOIN = L only + INNER

RIGHT JOIN = INNER + R only

FULL JOIN = L only + INNER + R only

ON vs WHERE -- The Critical Difference

This is a common trap. See 04-SQL-TRAPS.md Trap 2 for full detail.

Short rule:

- ON defines the join condition (evaluated during JOIN)

- WHERE filters after the join is complete

The trap:

```
```sql
-- WRONG: WHERE on right table kills LEFT JOIN silently
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'HR'; -- Carol (no dept) disappears!
-- CORRECT: filter in ON clause to keep unmatched left rows
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
 ON e.dept_id = d.dept_id
 AND d.dept_name = 'HR'; -- Carol stays, with NULL dept_name
```
```

ER Model -- Entity Relationship Basics

Core Components

Component
Represents
Symbol

Entity

A real-world object (Student, Employee)

Rectangle

Weak Entity

Exists only with a parent entity (OrderItem needs Order)

Double rectangle

Attribute

Property of an entity (Name, Age)

Oval

Relationship

Association between entities

Diamond

Weak Relationship

Relationship involving a weak entity

Double diamond

Types of Attributes

Type
Example
Symbol

Simple
Age, Salary
Plain oval

Composite
Name = FirstName + LastName
Oval with sub-ovals

Derived
Age (from DateOfBirth)
Dashed oval

Multi-valued
PhoneNumbers (can have many)
Double oval

Key attribute
StudentID (uniquely identifies entity)
Underlined in oval

Cardinality -- How Entities Relate

Type
Meaning
Real Example

1 : 1
One entity maps to exactly one other
Person Passport

1 : N
One entity maps to many others
Department Employees

M : N
Many entities map to many others
Students Courses

Participation Constraint

Type
Meaning
Notation
Example

Total (Mandatory)
Every entity MUST be in the relationship
Double line
Every Employee MUST belong to a Department

Partial (Optional)
Entity MAY or may not be in the relationship
Single line --
A Department MAY or may not have a Manager

Weak Entity -- Explained Simply
A weak entity cannot be uniquely identified on its own. It depends on a strong (owner) entity.
Example: OrderItem depends on Order
- OrderItem alone has no unique ID
- But {OrderID, ItemNumber} together identifies it uniquely
- ItemNumber here is called a partial key (discriminator)
Order (strong) OrderItem (weak)
 OrderID (PK) OrderID (FK) + ItemNumber (discriminator) = composite PK

Your 40-Second Script -- JOINS

"INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table plus matches from the right -- NULLs where there's no match. RIGHT JOIN is the mirror. FULL OUTER JOIN returns everything from both sides. CROSS JOIN is the cartesian product -- every row paired with every row. SELF JOIN joins a table with itself, useful for hierarchical data like employee-manager relationships."

Worked Examples -- EMPLOYEE & PROJECT Tables (from Ebook)
These two tables are used in all 4 JOIN examples below:
EMPLOYEE table:

EMP_ID
EMP_Name
CITY
SALARY
AGE

1
Angelina
Chicago
200000
30

2
Robert
Austin
300000
26

3
Christian
Denver
100000
42

4
Kristen
Washington
500000
29

5
Russell
Losangels
200000
36

PROJECT table:

PROJECT_NO
EMP_ID
DEPARTMENT

101
1
Testing

102
2
Development

103
3
Designing

104
4
Development

Note: EMP_ID 5 (Russell) has no project. No project has EMP_ID 5 or 6.

INNER JOIN -- Only Matched Rows
sql
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT
FROM EMPLOYEE
INNER JOIN PROJECT
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
Output:

EMP_NAME
DEPARTMENT

Angelina
Testing

Robert
Development

Christian
Designing

Kristen

Development

Russell (EMP_ID 5) is excluded -- no matching row in PROJECT.

LEFT JOIN -- All Employees, Matched Projects

```
sql
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT
FROM EMPLOYEE
LEFT JOIN PROJECT
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
Output:
```

| EMP_NAME | DEPARTMENT |
|----------|------------|
|----------|------------|

| | |
|----------|---------|
| Angelina | Testing |
|----------|---------|

| | |
|--------|-------------|
| Robert | Development |
|--------|-------------|

| | |
|-----------|-----------|
| Christian | Designing |
|-----------|-----------|

| | |
|---------|-------------|
| Kristen | Development |
|---------|-------------|

| | |
|---------|------|
| Russell | NULL |
|---------|------|

| | |
|-------|------|
| Marry | NULL |
|-------|------|

Russell and Marry appear with NULL -- they're in EMPLOYEE but not in PROJECT.

RIGHT JOIN -- All Projects, Matched Employees

```
sql
SELECT EMPLOYEE.EMP_NAME, PROJECT.DEPARTMENT
FROM EMPLOYEE
RIGHT JOIN PROJECT
ON PROJECT.EMP_ID = EMPLOYEE.EMP_ID;
Output:
```

| EMP_NAME | DEPARTMENT |
|----------|------------|
|----------|------------|

| | |
|----------|---------|
| Angelina | Testing |
|----------|---------|

| | |
|--------|-------------|
| Robert | Development |
|--------|-------------|

Christian
Designing

Kristen
Development

All rows from PROJECT table appear. Employees with no project are excluded.

FULL JOIN -- Everything From Both Tables

```
sql
SELECT table1.column1, table1.column2
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

Returns all rows from both tables. NULL where no match exists on either side.

FULL JOIN = LEFT JOIN + RIGHT JOIN (combined, deduped).

Follow-Up Questions (Expect These)

Q: What's the difference between JOIN and subquery -- when to prefer which?

JOINS are generally faster and more readable for combining tables. Subqueries are cleaner when the inner result is independent or used for filtering.

Q: Can you JOIN on multiple columns?

Yes: ON a.col1 = b.col1 AND a.col2 = b.col2

Q: What is a natural join?

Automatically joins on all columns with the same name. Avoided in practice -- fragile if schema changes.

Q: What is an equi-join vs non-equi-join?

Equi-join uses = in the ON clause. Non-equi-join uses <, >, BETWEEN, etc.